

Sistemas Operativos: Práctica 1

Shaofan Xu Javier Maria Santa

April 9, 2026

1 Introducción

En esta práctica el objetivo es implementar códigos en C que dado una función Hash $\text{pow} : \mathbb{Z} \rightarrow \mathbb{Z}/9997697\mathbb{Z}$ con $x \mapsto (X \times x + y) \bmod 9997697$ y la imagen, debemos sacar su correspondiente x . Donde aprovechamos los conocimientos de procesos e hilos para simular la minería de un blockchain.

2 Requisitos cumplidos y no cumplidos

2.1 Requisitos cumplidos

Se han implementado correctamente los siguientes objetivos principales de la práctica:

- **Control de salida (a):** Impresión exacta de los mensajes de estado (*“Miner exited with status 0”*, *“Logger exited with status 0”*, etc.) según las especificaciones.
- **Gestión de hilos y concurrencia (b):** Como sabemos que el rango de la preimagen está en $[0, 9997697]$ (para simplificar el problema), creamos N hilos mediante `pthread_create` para dividir el espacio de búsqueda en N , para cada uno de los hilos se encarga de cada parte de espacio de búsqueda, una vez un hilo se ha encontrado la solución, los otros inmediatamente tienen que parar. Así, sucesivamente para cada ronda.
- **Gestión de procesos (c):** Creación del proceso hijo Registrador mediante `fork ()` a partir de un proceso padre que denominamos Minero, donde este se encarga de crear hilos para buscar de manera a fuerza bruta la preimagen (x) de una imagen dada (y). Y Registrador se encarga de guardar los resultados en un fichero.
- **Comunicación IPC (c):** Se realiza a través de tuberías (pipe), donde establecemos 2 tuberías: Minero a Registrador y Registrador a Minero, y cerrando los lados que no se usen.
 - Minero a Registrador (pipe_miner_reg): Para que el Minero manda a Registrador el resultado de cada ronda de búsqueda. Y el Registrador recibe los resultados e imprime en un fichero “ppid.log” con el formato dado. Este proceso, se continúa hasta que el Minero manda una señal de terminación (-1 como solución en nuestro caso)

- Registrador a Minero (`pipe_reg_miner`): El Registrador manda una confirmación (1 en nuestro caso) de que se ha recibido correctamente por la tubería. Una vez el Minero recibe la confirmación se sigue con la siguiente ronda.

- **Control de errores:**

- **System Call** Hemos tenido en cuenta los casos en la que se usan llamadas al sistema como *malloc*, *pipe*, *pthread_create*, etc, así como sus controles de errores y liberación.
- **Gestión adecuada de hilos:** El proceso principal Minero espera a todos los hilos creados mediante `pthread_join()` antes de enviar los resultados de la ronda al Registrador.
- **Gestión adecuada de proceso:** Cuando termina todas las rondas el proceso principal Minero espera al proceso hijo Registrador mediante `wait(&status)`.
- **Gestión adecuada de tuberías:** Durante la ejecución de programas, cada uno de los procesos se cierran el lado de tubería que no lo van a utilizar, y una vez, alcanzado el máximo de las rondas se cierran todas las tuberías.

2.2 Requisitos no cumplidos

Todos los requisitos obligatorios especificados en el enunciado han sido implementados y funcionan correctamente.

3 Pruebas: Funcionamiento y Rendimiento

3.1 Entorno de ejecución

Las pruebas y mediciones se han llevado a cabo bajo las siguientes condiciones de hardware y software para garantizar la consistencia de los resultados:

- Sistema Operativo: WSL2 (Ubuntu) sobre Windows 11.
- Hardware: 16 GB de memoria RAM.
- Herramienta de medición: Comando `time` de la shell de Linux.

3.2 Pruebas de Funcionamiento

Se han ejecutado diversas pruebas para asegurar el correcto funcionamiento del programa.

3.2.1 Casos normales

Se ejecutó el programa con parámetros correspondientes (ej.target inicial 100, 10 rondas y 4 hilos):

```
./miner 100 10 4
```

El programa generó correctamente el archivo `.log` con la estructura especificada(Id, ronda, target, solución, etc), alternando la validación (accepted/rejected). Al finalizar, se imprime en la terminal exactamente los mensajes requeridos:

```
Solution accepted: 100 —> 2813411
Solution rejected: 2813411 —> 4952263
Solution rejected: 4952263 —> 1751421
Solution rejected: 1751421 —> 4619098
Solution accepted: 4619098 —> 9959801
Solution rejected: 9959801 —> 297242
Solution accepted: 297242 —> 7642589
Solution rejected: 7642589 —> 1566555
Solution rejected: 1566555 —> 7848135
Solution rejected: 7848135 —> 5915685
Logger exited with status 0
Miner exited with status 0
```

3.2.2 Casos de errores

Se probaron situaciones anómalas como:

- **Falta de argumentos:** Al ejecutar `./miner` sin parámetros, el programa detecta el error, muestra el uso correcto “*Usage ./miner < TARGET_INI > < ROUNDS > < N_THREADS >*” e imprime “*Miner exited unexpectedly*”.

3.3 Pruebas de Rendimiento

Para evaluar la eficiencia de la paralelización implementada, se han realizado pruebas de rendimiento variando el número de hilos de ejecución. Estas

pruebas permiten analizar cómo el sistema escala al dividir el espacio de búsqueda entre múltiples unidades de procesamiento.

3.3.1 Pruebas realizadas

Se ejecutó el comando `./miner 0 10 N` (donde N es el número de hilos) manteniendo fijos el objetivo inicial (0) y el número de rondas (10).

```
dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I1/I1/SO/practical$ time ./miner 0 10 1
Solution accepted: 0 --> 8916551
Solution rejected: 8916551 --> 5970796
Solution accepted: 5970796 --> 937352
Solution accepted: 937352 --> 1835314
Solution accepted: 1835314 --> 9331340
Solution accepted: 9331340 --> 2986686
Solution accepted: 2986686 --> 7299854
Solution accepted: 7299854 --> 3981310
Solution accepted: 3981310 --> 8021305
Solution rejected: 8021305 --> 4673876
Logger exited with status 0
Miner exited with status 0

real    0m0.166s
user    0m0.075s
sys     0m0.000s
dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I1/I1/SO/practical$ time ./miner 0 10 2
Solution accepted: 0 --> 8916551
Solution rejected: 8916551 --> 5970796
Solution accepted: 5970796 --> 937352
Solution accepted: 937352 --> 1835314
Solution accepted: 1835314 --> 9331340
Solution accepted: 9331340 --> 2986686
Solution accepted: 2986686 --> 7299854
Solution accepted: 7299854 --> 3981310
Solution accepted: 3981310 --> 8021305
Solution rejected: 8021305 --> 4673876
Logger exited with status 0
Miner exited with status 0

real    0m0.107s
user    0m0.046s
sys     0m0.007s
dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I1/I1/SO/practical$ time ./miner 0 10 4
Solution accepted: 0 --> 8916551
Solution rejected: 8916551 --> 5970796
Solution accepted: 5970796 --> 937352
Solution accepted: 937352 --> 1835314
Solution accepted: 1835314 --> 9331340
Solution accepted: 9331340 --> 2986686
Solution accepted: 2986686 --> 7299854
Solution accepted: 7299854 --> 3981310
Solution accepted: 3981310 --> 8021305
Solution rejected: 8021305 --> 4673876
Logger exited with status 0
Miner exited with status 0

real    0m0.074s
user    0m0.010s
sys     0m0.003s
dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I1/I1/SO/practical$ time ./miner 0 10 8
Solution accepted: 0 --> 8916551
Solution rejected: 8916551 --> 5970796
Solution accepted: 5970796 --> 937352
Solution accepted: 937352 --> 1835314
Solution accepted: 1835314 --> 9331340
Solution accepted: 9331340 --> 2986686
Solution accepted: 2986686 --> 7299854
Solution accepted: 7299854 --> 3981310
Solution accepted: 3981310 --> 8021305
Solution rejected: 8021305 --> 4673876
Logger exited with status 0
Miner exited with status 0

real    0m0.087s
user    0m0.094s
sys     0m0.000s
```

Figure 1: Resultados de ejecución del algoritmo POW con diferentes hilos.

3.3.2 Resultados Obtenidos

Los datos obtenidos se resumen en la siguiente tabla:

Hilos (N)	Tiempo Real (s)	Tiempo User (s)	Tiempo Sys (s)	Speedup (S_n)
1	0.166s	0.075s	0.000s	1.00x (Base)
2	0.107s	0.046s	0.007s	1.55x
4	0.074s	0.010s	0.003s	2.24x
8	0.087s	0.094s	0.000s	1.90x

Table 1: Tiempos de ejecución y Speedup en función del número de hilos

3.3.3 Análisis y Razonamiento

Tras analizar los datos de la tabla, se pueden extraer las siguientes conclusiones técnicas:

- Paralelización efectiva (1 a 4 hilos): Se observa una reducción notable en el tiempo Real a medida que aumentamos los hilos hasta 4. Esto confirma que la estrategia de fragmentar el intervalo POW_LIMIT permite que los hilos realicen búsquedas de fuerza bruta de forma independiente y concurrente, optimizando el uso de los núcleos de la CPU.
- Saturación y Overhead (8 hilos): Al incrementar el número de hilos a 8, el tiempo Real aumenta ligeramente a 0.087s y el tiempo User se eleva significativamente a 0.094s. Este fenómeno ocurre porque, para una carga de trabajo de solo 10 rondas, el coste temporal de crear, sincronizar y destruir 8 hilos mediante `pthread_create` y `pthread_join` supera el beneficio obtenido por la computación paralela.
- Número óptimo de hilos: Para la arquitectura del sistema utilizado (WSL2) y la carga de trabajo específica de la práctica, el número óptimo se sitúa en 4 hilos. En este punto se alcanza el mejor equilibrio entre el aprovechamiento de la CPU y la sobrecarga mínima del sistema operativo en la gestión de hilos y cambios de contexto.
- Consumo de CPU: El hecho de que el tiempo User (tiempo total de CPU) sea superior al tiempo Real en la prueba de 8 hilos refleja que el sistema está invirtiendo más esfuerzo en la gestión de la concurrencia que en la resolución efectiva del problema matemático.

4 Conclusión y Respuesta a las cuestiones del enunciado

Pregunta d) ¿Qué conclusiones se pueden tomar respecto al número óptimo de hilos?

Una vez concluido todos los trabajos realizados, podemos concluir que el número óptimo de hilos coincide con el número de procesadores disponible para máquina.

Como se observa en la tabla, el máximo rendimiento se alcanza exactamente al utilizar 4 hilos. Al incrementar el número a 8 hilos, el rendimiento disminuye.

Ya que si creamos más hilos que el número del procesador, no es posible ejecutar los hilos de forma simultánea. El planificador del sistema operativo se va a obligar intercalando su ejecución.